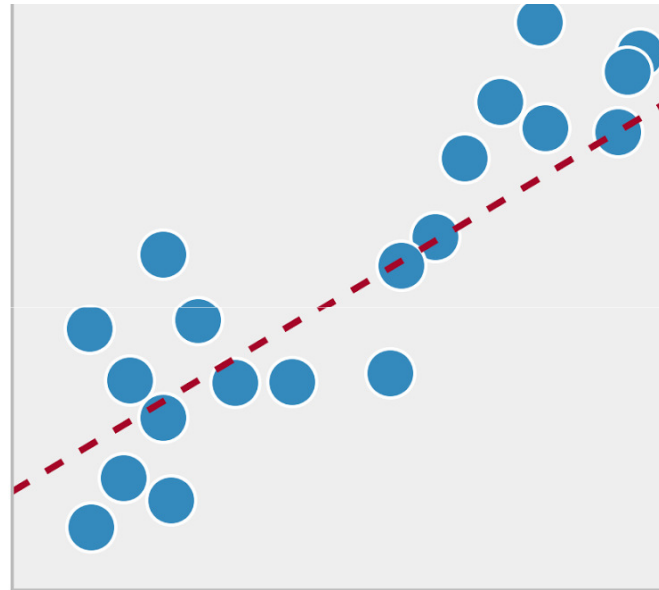




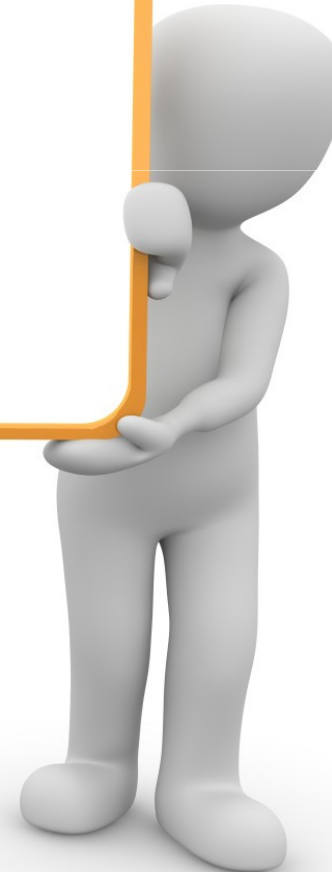
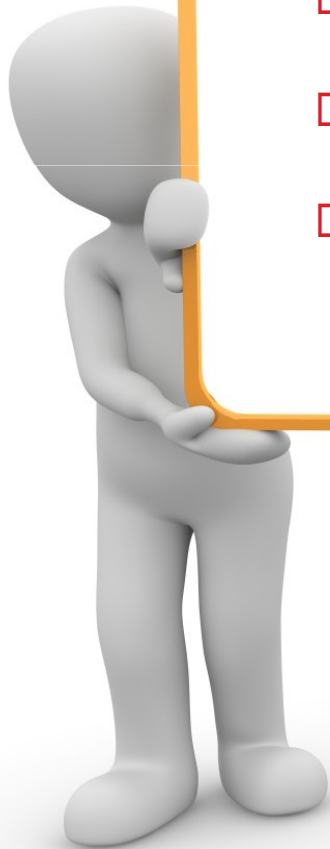
Régression linéaire

Enseignante : Dr. Rim Mahouachi

Plan

2

- ❑ Régression univariée
- ❑ Régression multivariée
- ❑ Régression polynomiale



3

Régression univariée

Principe

4

- C'est un algorithme prédictif supervisé
- Entrée : une variable prédictive x
- Sortie : trouver une fonction de prédiction $h(x)$
 - ▣ Cette fonction sera une droite qui s'approchera le plus possible des données d'apprentissage.
 - ▣ La fonction de prédiction étant une droite, elle s'écrira mathématiquement sous la forme :

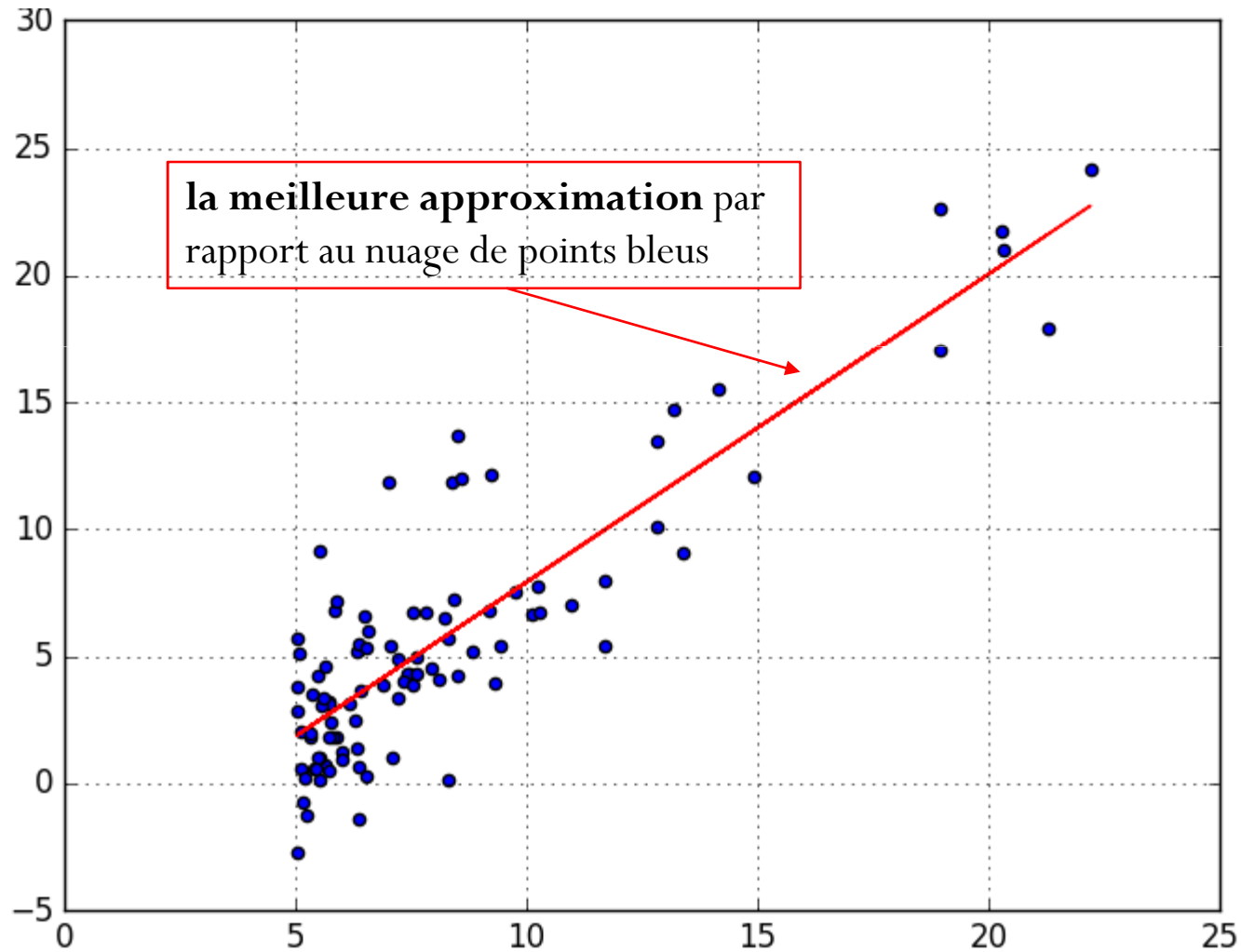
$$h(x) = \theta_0 + \theta_1 x$$

les coefficients de la droite



Principe

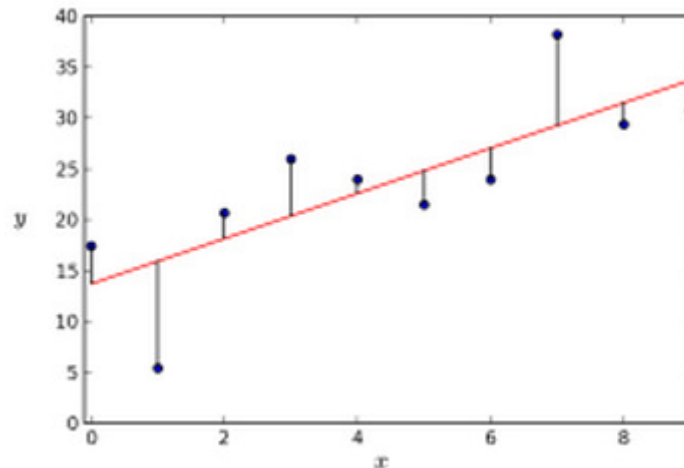
5



Principe

6

- Comment on calcule les valeurs de θ_0 et θ_1 ?
- La droite en rouge tente d'approcher le plus de points possibles (en réduisant l'écart avec ces derniers).
 - ▣ En d'autres termes, elle **minimise au maximum l'erreur globale**.



Principe

7

- Le but est de trouver un couple (θ_0, θ_1) optimal tel que $h(x)$ soit le plus proche possible de y (la valeur qu'on essaie de prédire), pour tous les couples (x, y) qui forment l'ensemble de données d'apprentissage
- La fonction h va essayer de transformer au mieux $x^{(i)}$ en $y^{(i)}$ tel que $h(x^{(i)}) \approx y^{(i)}$

La fonction de coût d'erreur

8

- On définit l'*erreur unitaire* entre une valeur observée $y^{(i)}$ et une valeur prédite $h(x^{(i)})$, comme suit :

$$(h(x^{(i)}) - y^{(i)})^2$$

- Trouver le meilleur couple (θ_0, θ_1) revient à minimiser le coût global des erreurs unitaires qui se définit comme suit :

$$\sum_{i=1}^N (h(x^{(i)}) - y^{(i)})^2$$

N : taille du training set,
de l'échantillon

- La fonction de coût est définie comme suit :

$$J(\theta_0, \theta_1) = \frac{1}{2N} \sum_{i=1}^N (h(x^{(i)}) - y^{(i)})^2$$



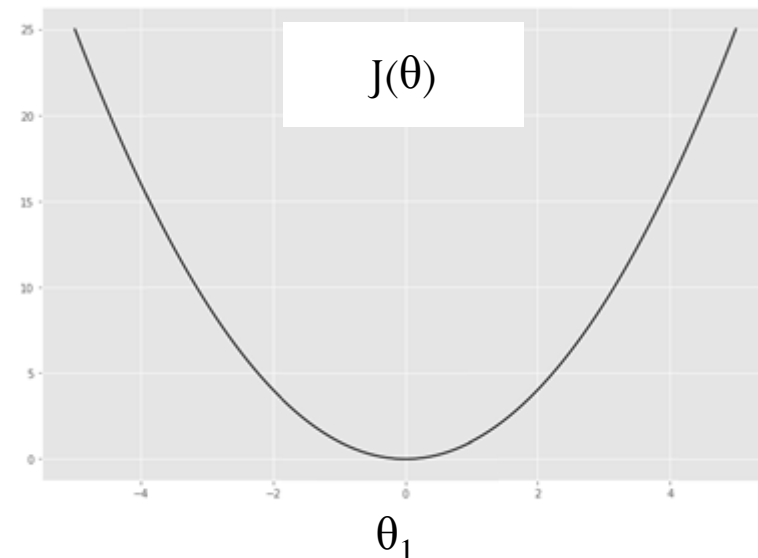
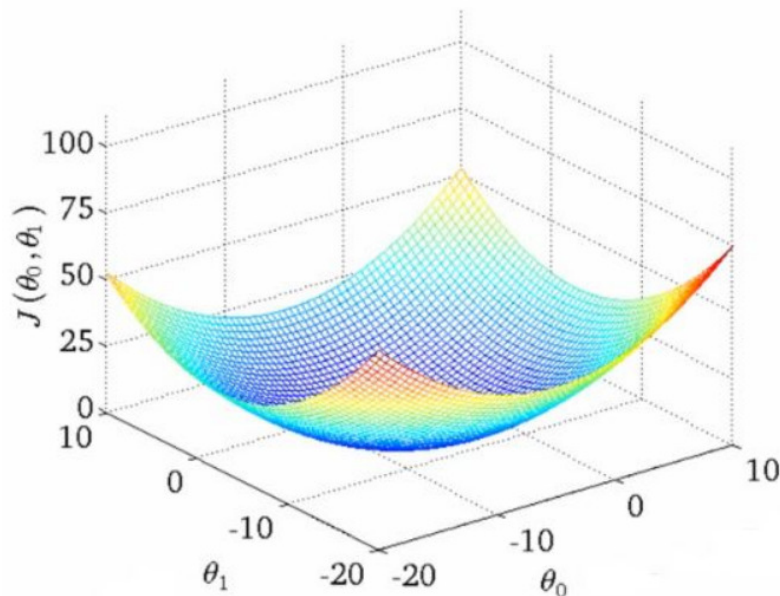
La fonction de coût d'erreur

9

- Trouver le meilleur couple (θ_0, θ_1) revient à minimiser la fonction de coût définie comme suit :

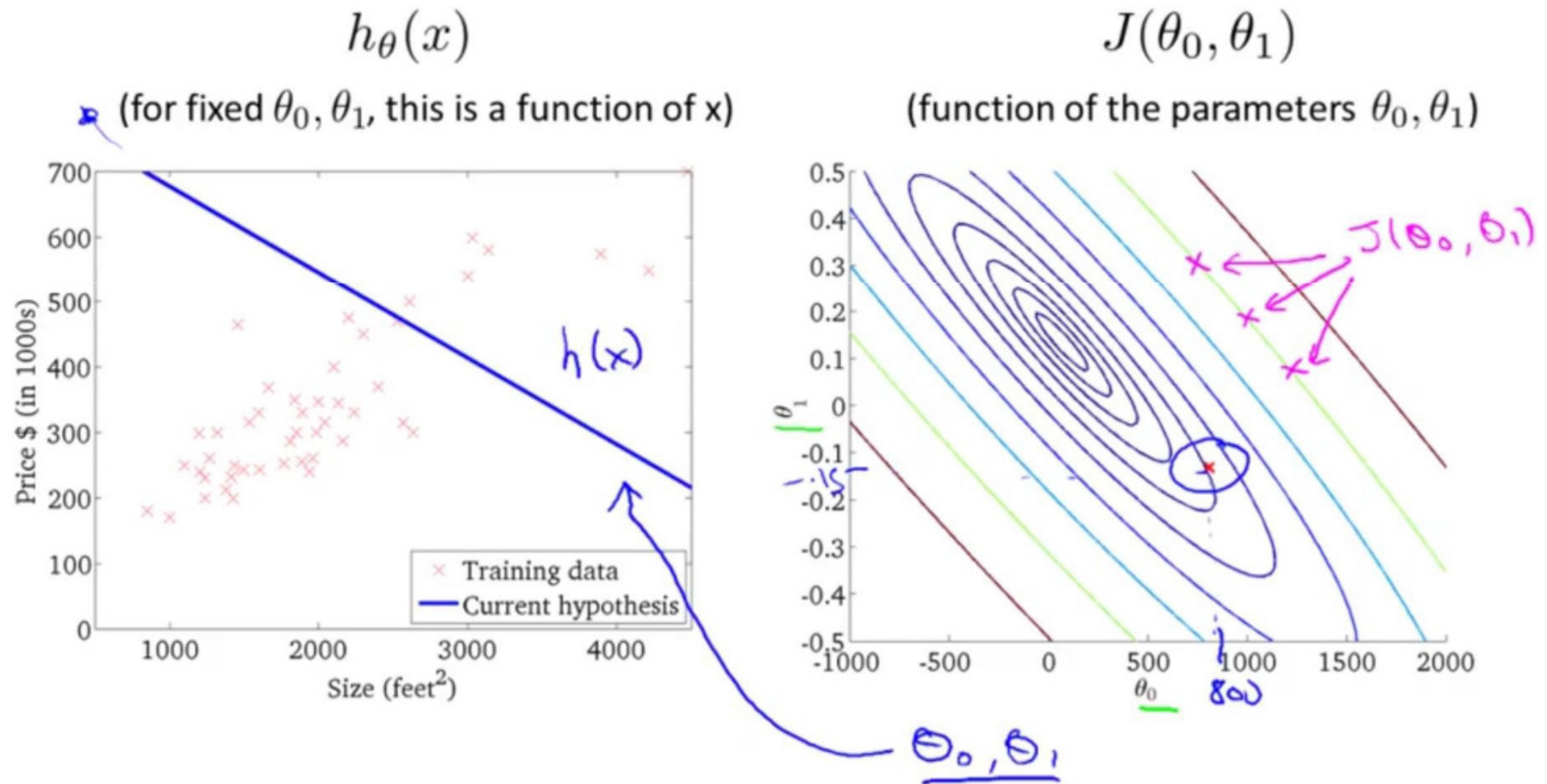
$$J(\theta_0, \theta_1) = \frac{1}{2N} \sum_{i=1}^N (h(x^{(i)}) - y^{(i)})^2$$

N : taille du training set, de l'échantillon



La fonction de coût d'erreur

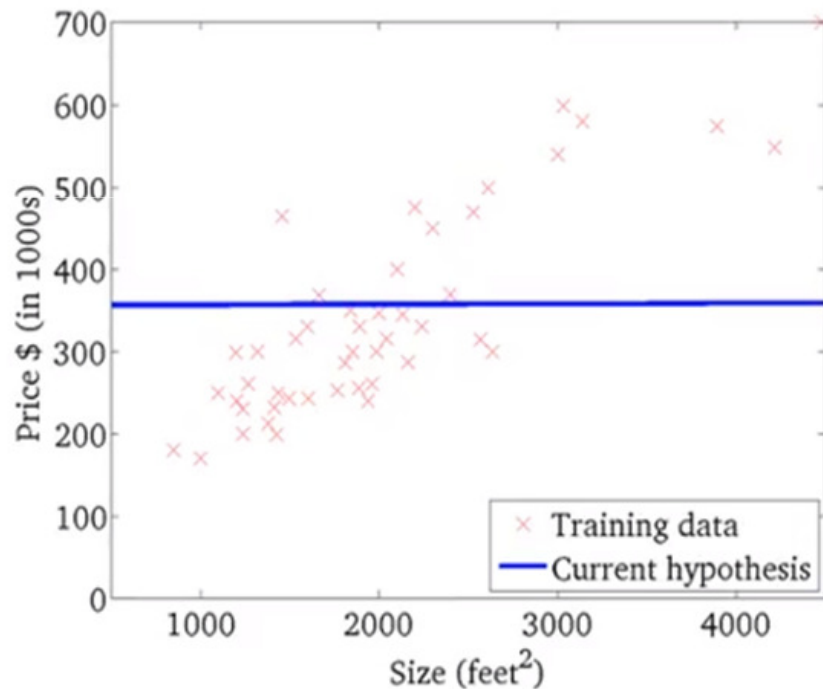
10



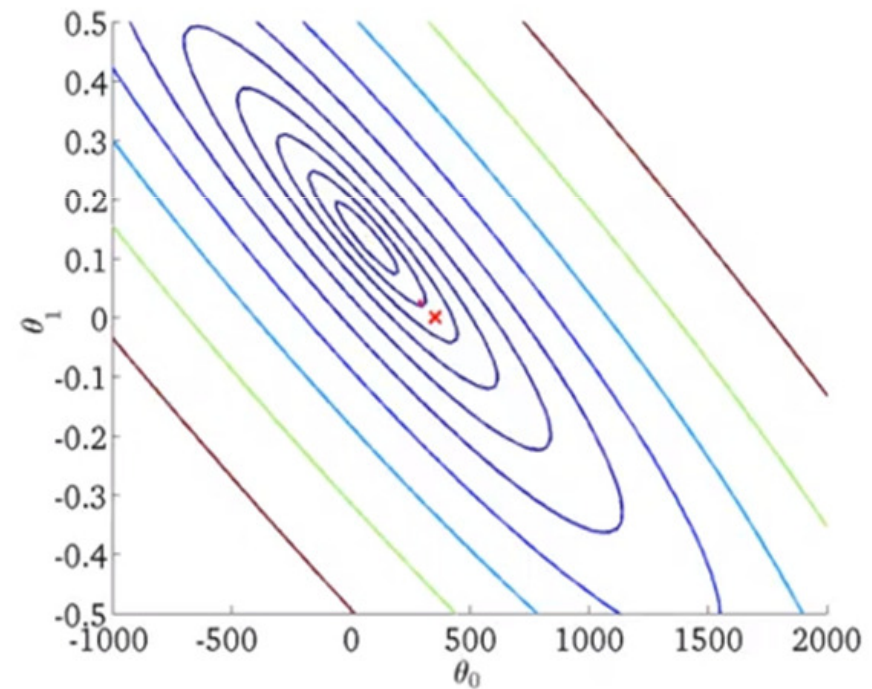
La fonction de coût d'erreur

11

(for fixed θ_0, θ_1 , this is a function of x)



(function of the parameters θ_0, θ_1)



Méthode de descente de gradient

12

- Une façon de calculer le minimum de la fonction de coût
 - ▣ utiliser l'algorithme de descente du gradient (Gradient descent).
 - ▣ C'est un algorithme itératif qui va changer, à chaque itération, les valeurs de θ_0 et θ_1 jusqu'à trouver le meilleur couple possible.

Algorithme de descente de gradient

13

- Initialiser aléatoirement les valeurs de θ_0 et θ_1
- Répéter jusqu'à convergence au minimum global de la fonction de coût:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \text{ pour } j = 1 \text{ et } j = 0$$

- Retourner θ_0 et θ_1



- α est le taux d'apprentissage (**learning rate**) qui fixe la “grandeur” du pas de chaque itération du Gradient Descent
- $\alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ est la **dérivée partielle** pour chacun des termes

Algorithme de descente de gradient

14

- Initialiser aléatoirement les valeurs de θ_0 et θ_1
- Répéter jusqu'à convergence au minimum global de la fonction de coût:

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

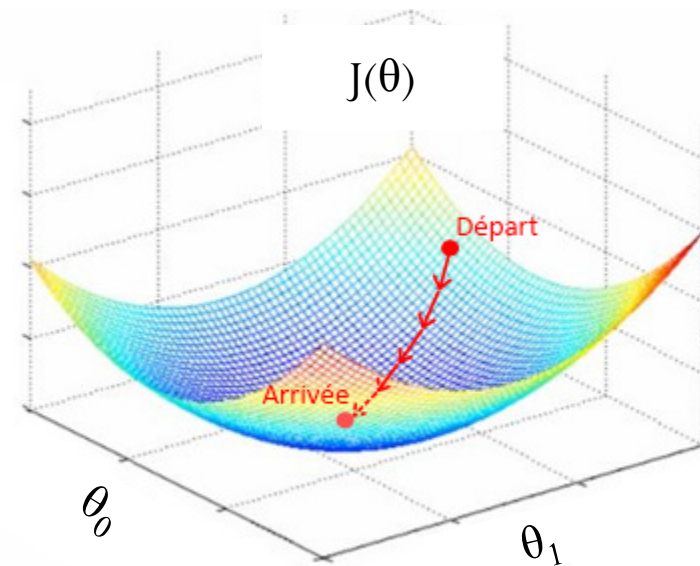
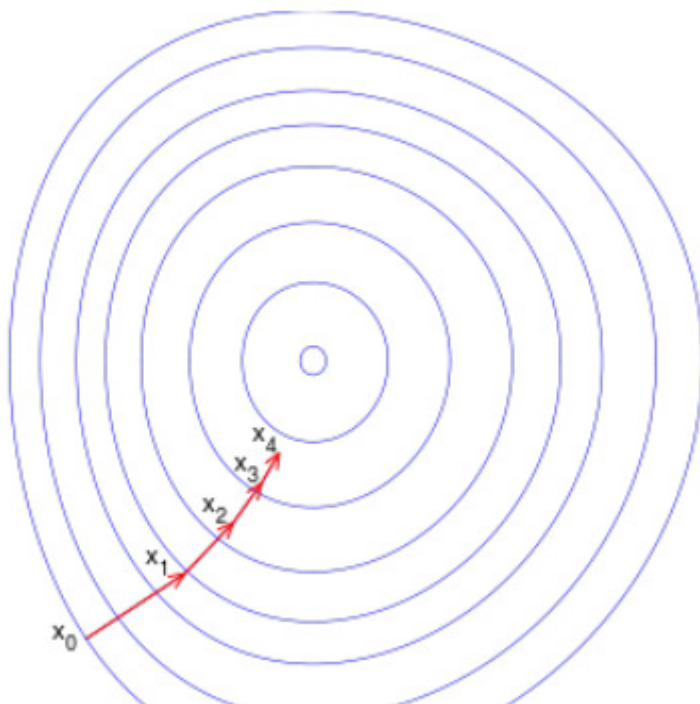
$$\theta_1 := \theta_1 - \alpha \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

- Retourner θ_0 et θ_1

Méthode de descente de gradient

15

- A chaque itération, l'algorithme avancera d'un pas et trouvera un nouveau couple (θ_0, θ_1)
 - ▣ à chaque itération, **le coût d'erreur global se réduira**



Méthode de descente de gradient

16

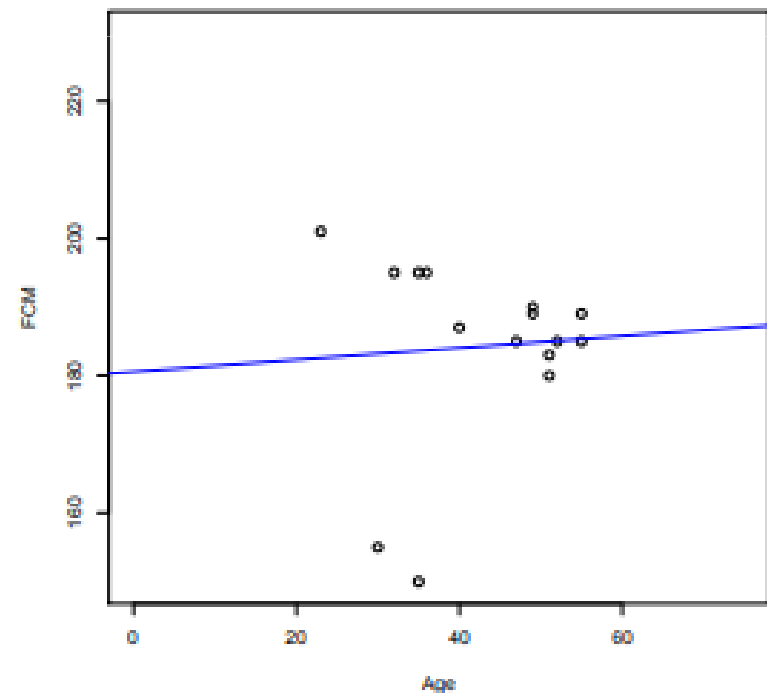
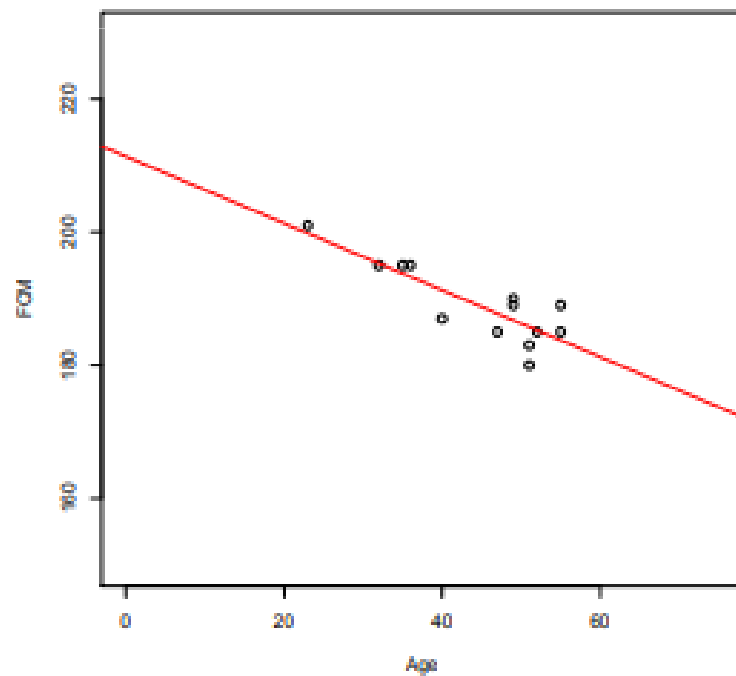
- Le vecteur gradient de taille $(p+1, 1)$ est composé des dérivées partielles de J par rapport à chaque paramètre du modèle
- Manipuler un vecteur de taille $(p+1, 1)$ plutôt qu'une matrice $(p+1, p+1)$, voilà tout l'intérêt de la descente de gradient pour les grandes dimensions



Observations aberrantes / influentes

17

- Effet important sur l'estimation de la droite de régression
- Mauvais ajustement aux données



18

Régression multivariée

Régression linéaire multivariée

19

- Exemple : prédiction du prix d'une voiture
 - ▣ Le prix est la **variable cible**,
 - ▣ **les variables prédictives** peuvent être : nombre de kilomètres au compteur, le nombre de cylindres, nombre de portes...etc.

- On parle de **régression linéaire multivariée**

Régression linéaire multivariée

20

- Pour p variables d'entrée (features), la fonction prédictive s'écrit sous la forme:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$$

- Et pour minimiser J , nous pouvons utiliser la méthode générale de descente de gradient:

- ▣ Répéter

$$\theta_j := \theta_j - \alpha \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad \text{pour } j = 0 \dots p$$

- ▣ J étant toujours convexe, pas de problème de minima locaux



Régression linéaire multivariée

21

- Et pour minimiser J , nous pouvons utiliser la méthode générale de descente de gradient:
 - ▣ Répéter

$$\begin{aligned}\theta_0 &:= \theta_0 - \alpha \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \\ \theta_1 &:= \theta_1 - \alpha \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)} \\ \theta_2 &:= \theta_2 - \alpha \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)} \\ &\dots\end{aligned}$$

Equation normale

22

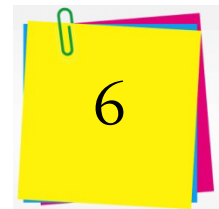
- La fonction coût sous forme vectorielle s'écrit:

$$J(\theta) = \frac{1}{2N} \sum (X \cdot \theta - Y)^2$$

Avec :

$$\theta^T = (\theta_0, \theta_1, \theta_2, \dots, \theta_p)$$

$$X = \begin{pmatrix} -(X^{(1)})^T - \\ -(X^{(2)})^T - \\ \vdots \\ -(X^{(N)})^T - \end{pmatrix} \quad \text{et} \quad Y = \begin{pmatrix} Y^{(1)} \\ Y^{(2)} \\ \vdots \\ Y^{(N)} \end{pmatrix}$$



Equation normale

23

- On a un problème de minimisation par rapport aux paramètres θ :

$$J(\theta) = \frac{1}{2N} \sum (X \cdot \theta - Y)^2$$

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{2}{2N} X^T (X\theta - y) = 0$$

$$X^T X \theta = X^T y$$

Il existe une solution exacte :

$$\theta = (X^T X)^{-1} X^T y$$

Problème

24

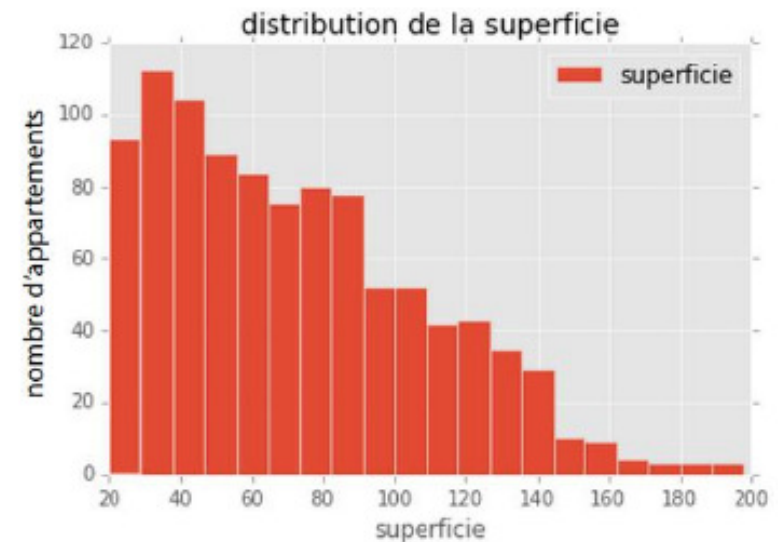
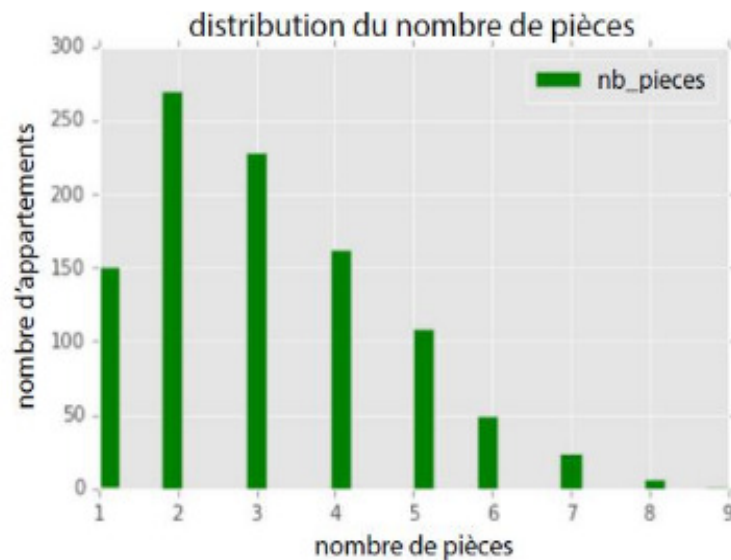
$$\theta = (X^T X)^{-1} X^T y$$

- Nécessite la manipulation de la matrice $(X^T X)$ de taille $(p+1, p+1)$, p nombre de variables explicatives
- ingérable dès que p est élevé (millier de variables, grandes dimensions)

Normalisation

25

- Exemple :
 - ▣ prédire les prix d'appartements tunisiens
 - ▣ 2 variables explicatives:
 - X1 : Nombre de pièces
 - X2 : Superficie



Normalisation

26

- Problème : convergence de l'algorithme de descente de gradient va demander plus d'itérations
- Solution : normalisation
 - ▣ Exemple : normalisation MinMax (entre 0 et 1)

$$X_{std} = \frac{X - \min(X)}{\max(X) - \min(X)}$$



27

Régression polynomiale

Régression polynomiale

28

- Dans le cas univarié:

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_M x^M = \sum_{j=0}^M \theta_j x^j$$

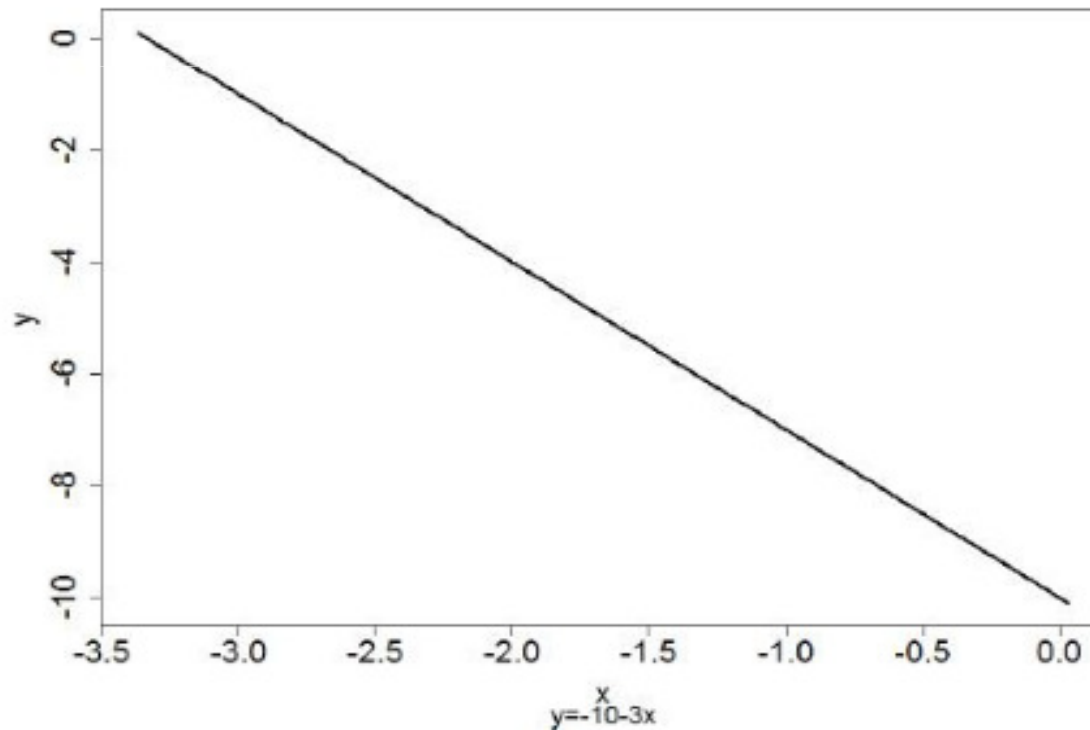
- Exemple : modèle polynomiale de degré $M = 2$ à 2 variables explicatives

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_2 + \theta_4 x_2^2$$

Régression polynomiale

29

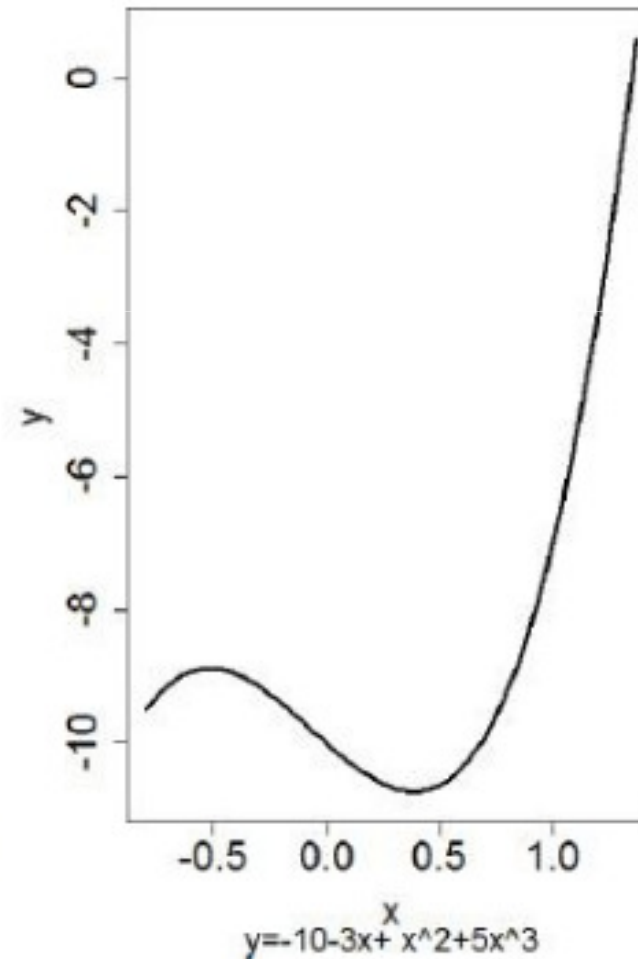
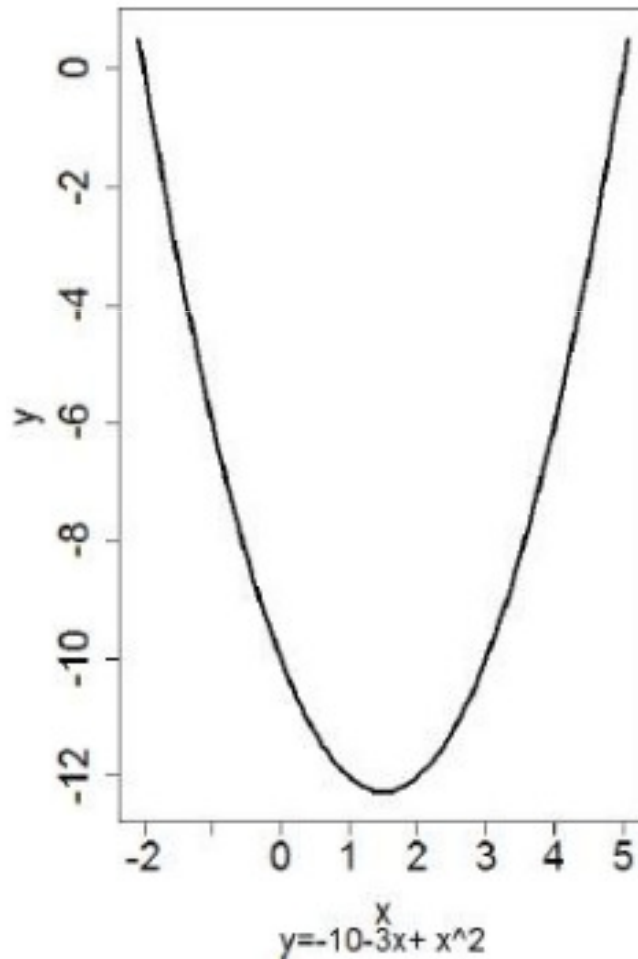
- Exemple de polynôme de degré 1
 - ▣ La régression linéaire n'est qu'une régression polynomiale de degré 1



Régression polynomiale

30

En rajoutant un terme en $M = 2$ ou en $M = 3$:

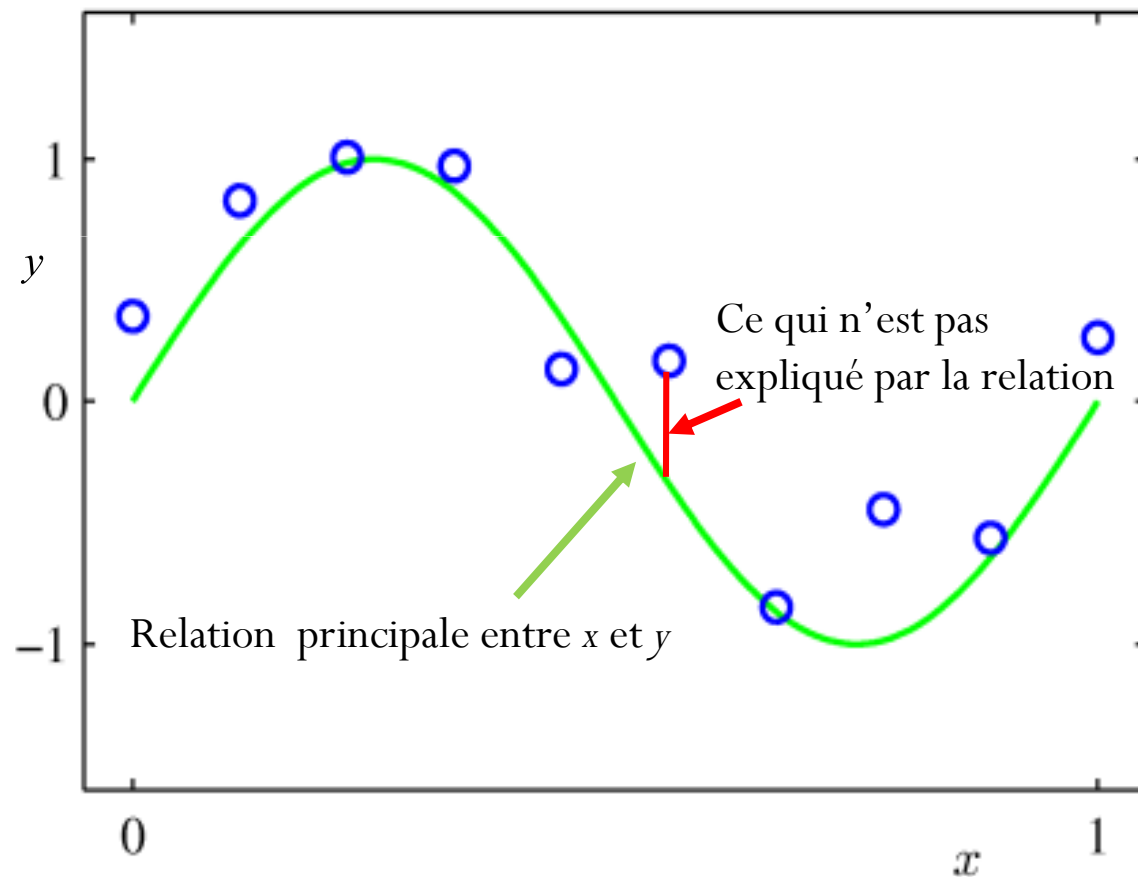


Chaque nouvel ordre va permettre d'ajouter un « pli » à la courbe

Exemple

31

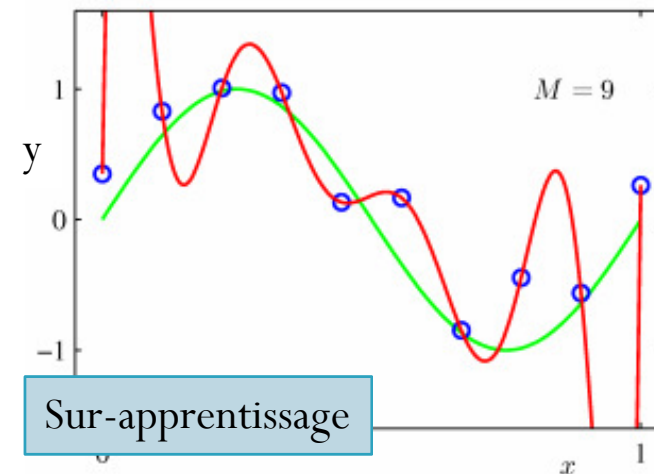
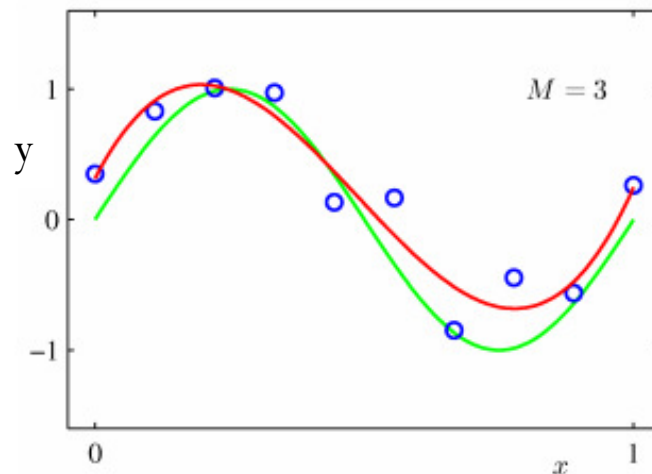
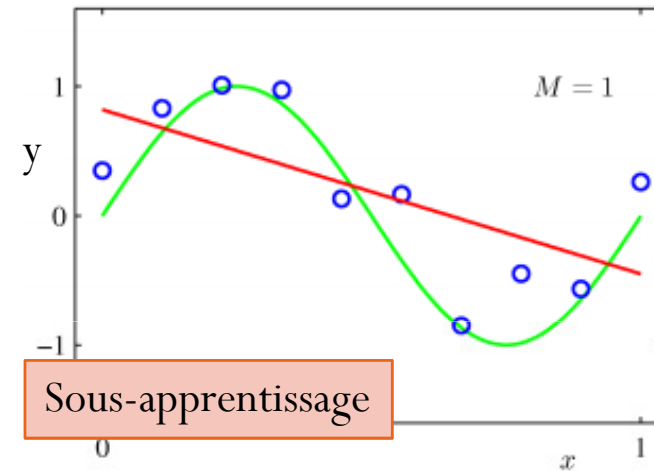
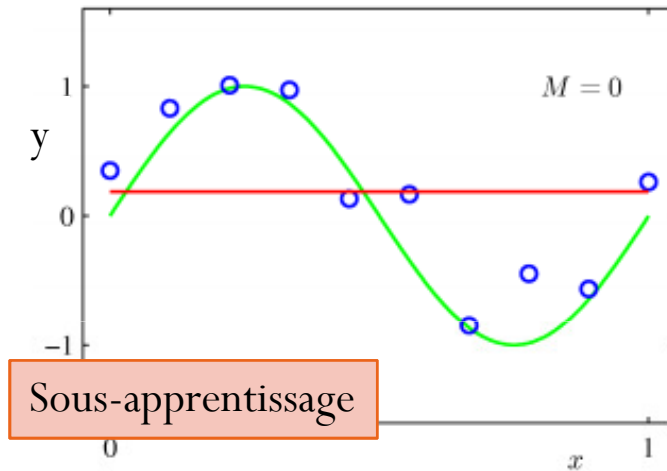
Exemple : Estimation d'une courbe (1-D)



Sur et sous-apprentissage

32

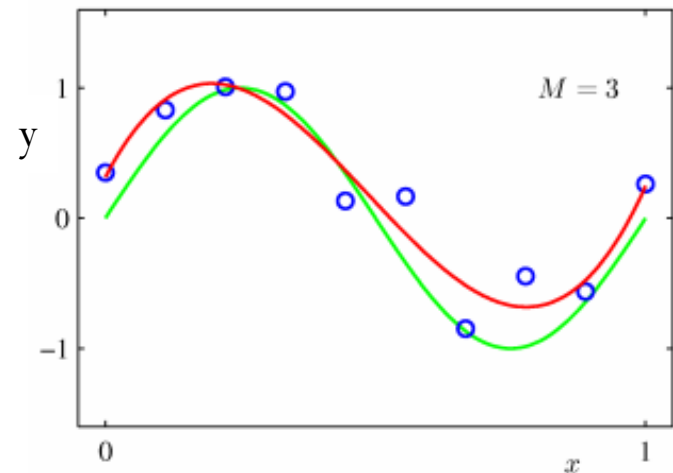
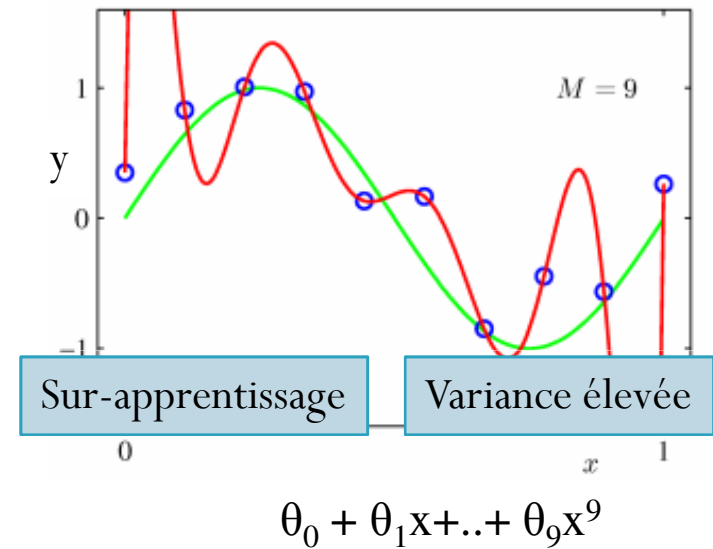
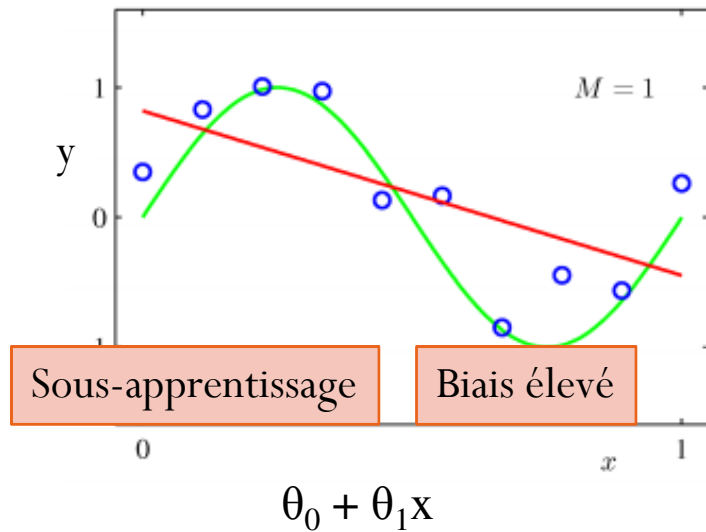
Comment choisir M ?



[C. Bishop, Pattern recognition and Machine learning, 2006]

Compromis Biais - Variance

33



Compromis Biais - Variance

34

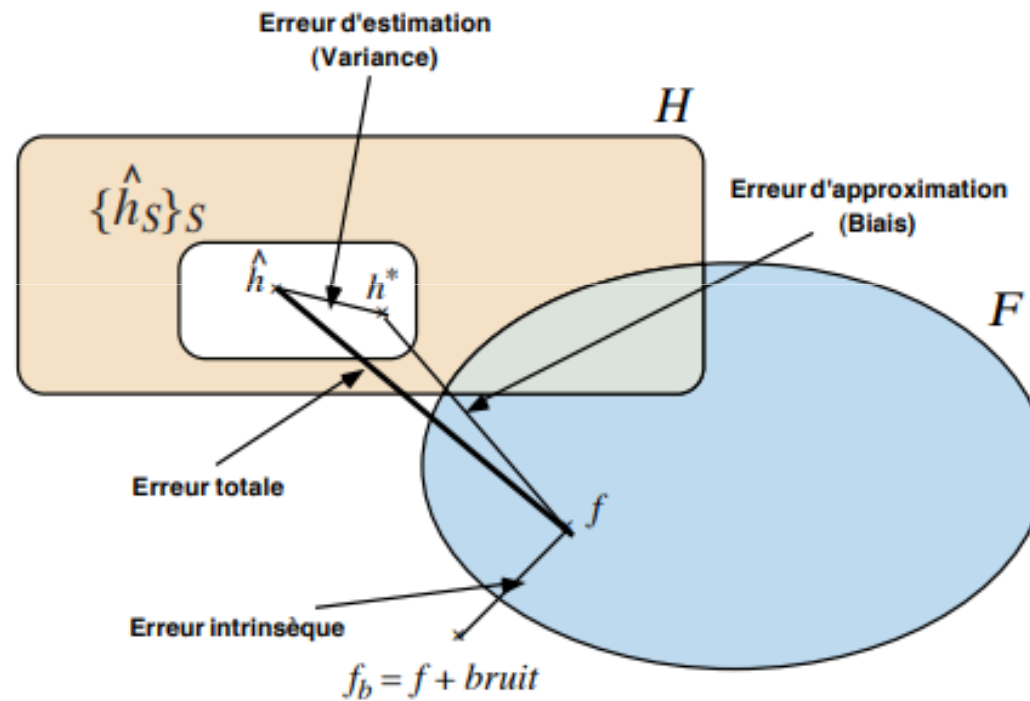
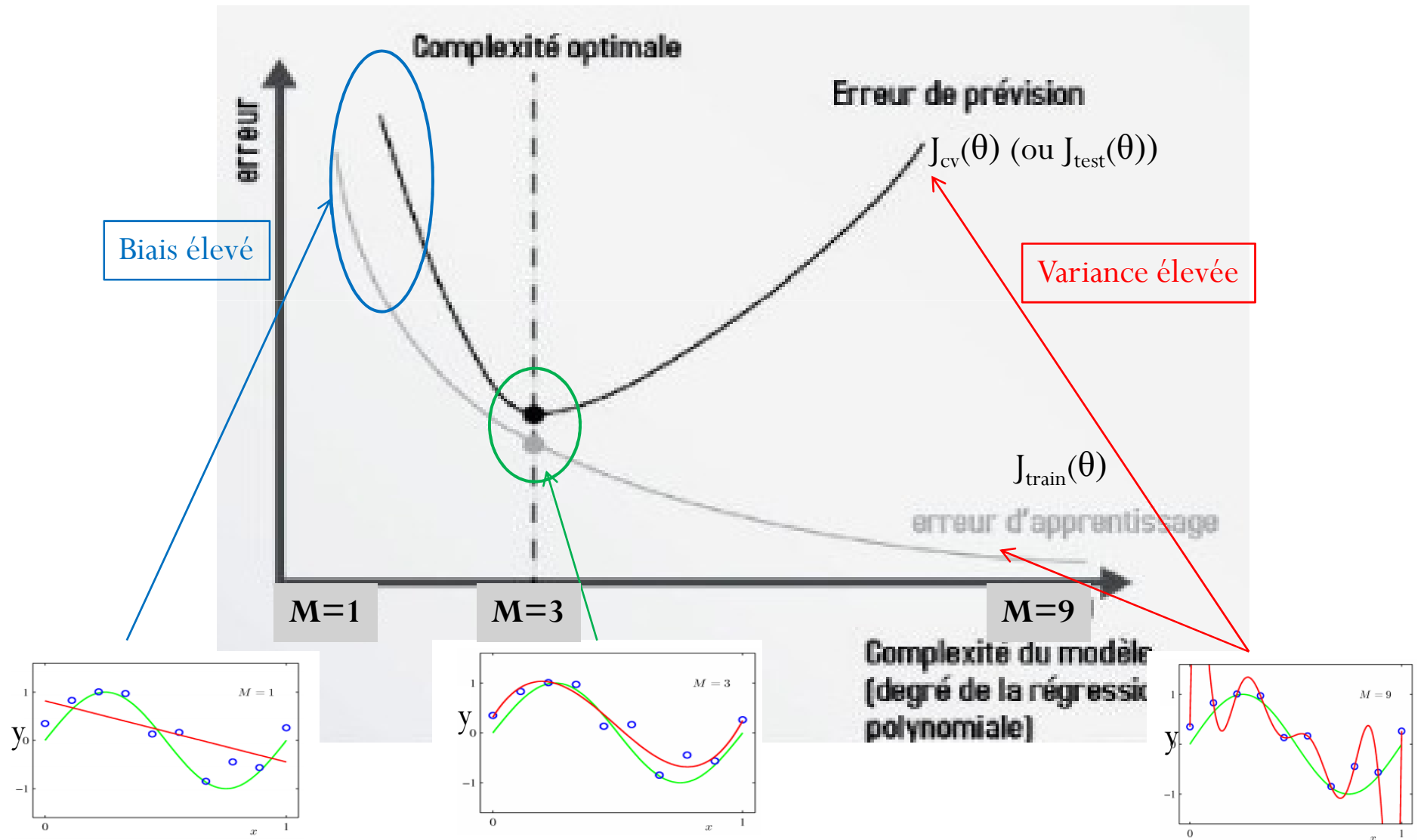


FIG.: Les différents types d'erreurs.

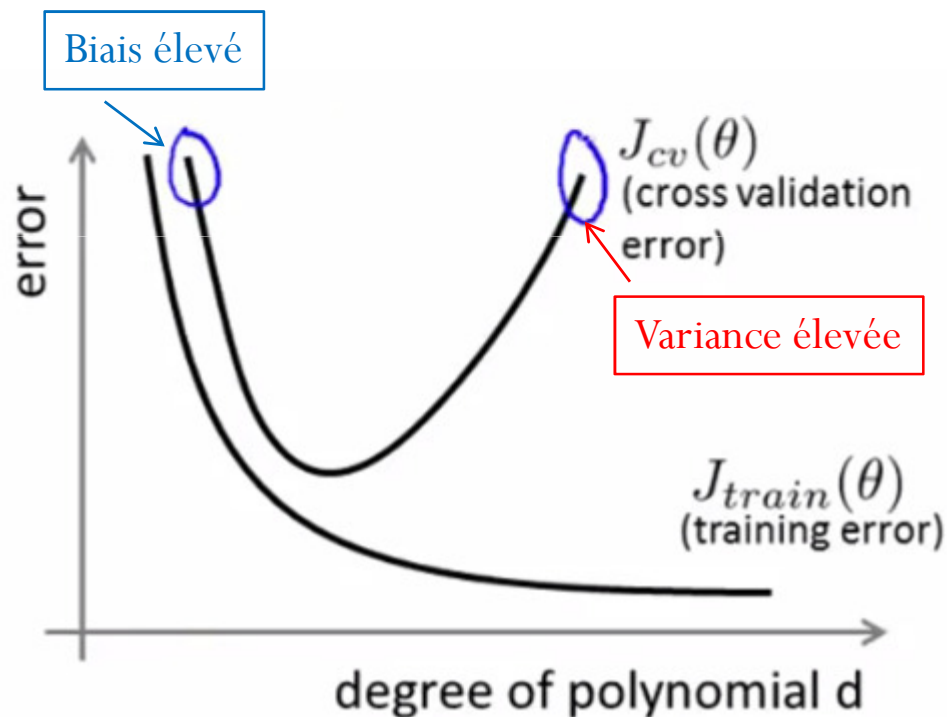
Compromis Biais - Variance

35



Compromis Biais - Variance

36



Sous-apprentissage (Biais élevé)
 $J_{train}(\theta)$ est élevée

Sur-apprentissage (Variance élevée)
 $J_{train}(\theta)$ est basse
 $J_{train}(\theta) \ll J_{cv}(\theta)$



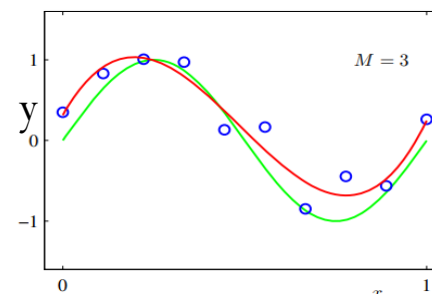
Comment éviter le sur-apprentissage ?

37

□ Solutions possibles

▣ Réduire le nombre de variables (features)

- Sélectionner manuellement les variables à garder
- **Sélection de modèle** : trouver la meilleure valeur de M
 - qui permet de retrouver la tendance générale de la relation entre x et y , sans le bruit
 - qui va permettre de bien généraliser à de nouvelles entrées



▣ Régularisation

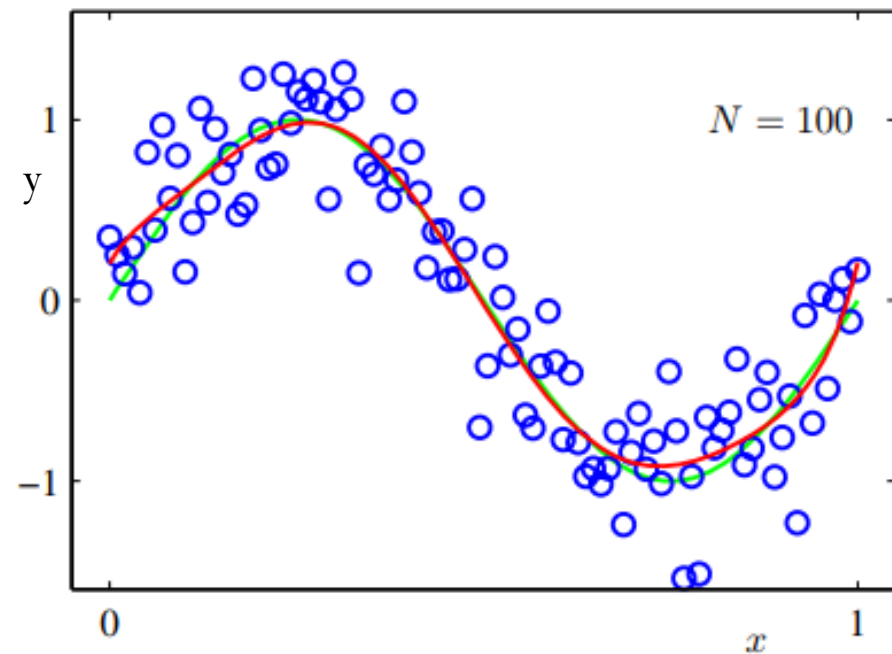
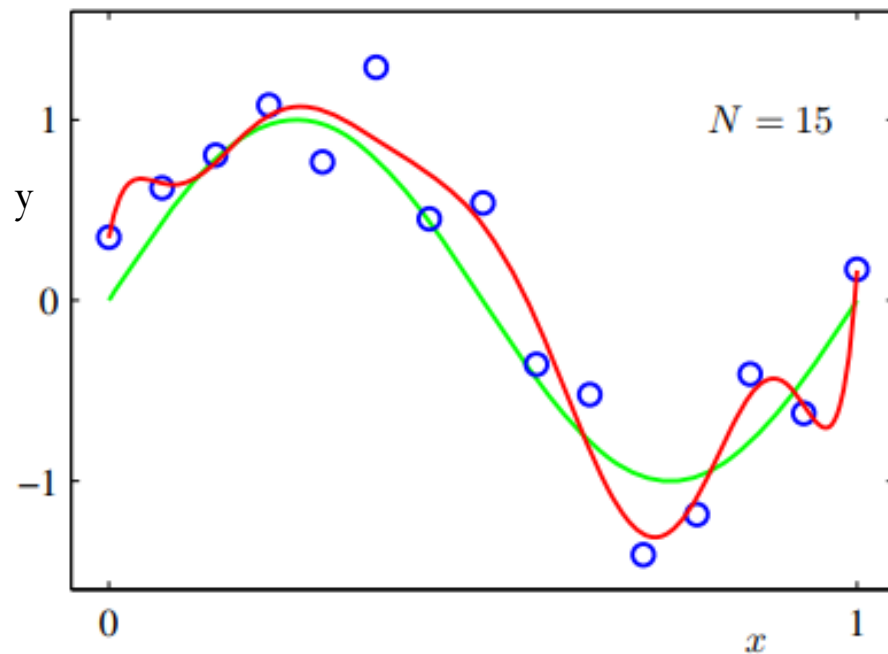
- Laisser tous les attributs mais réduire les valeurs des paramètres θ_j
- Marche bien quand on a beaucoup de variables et que chacune d'entre elles contribue un peu à la prédiction de y

Big data

38

□ Généralisation vs. quantité de données

- Plus la quantité de données d'entraînement augmente, plus le modèle entraîné va bien généraliser (et la variance va diminuer)



Régularisation

39

□ Principe :

■ On minimise à la fois la perte et la complexité

lambda (ou taux de régularisation)

$$J(\theta) = \frac{1}{2^N} \left[\sum_{i=1}^N (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \underbrace{\lambda \sum_{j=1}^M \theta_j^2}_{\text{L}_2 \text{ : Facteur de régularisation}} \right]$$

L₂ : Facteur de régularisation

Régularisation

40

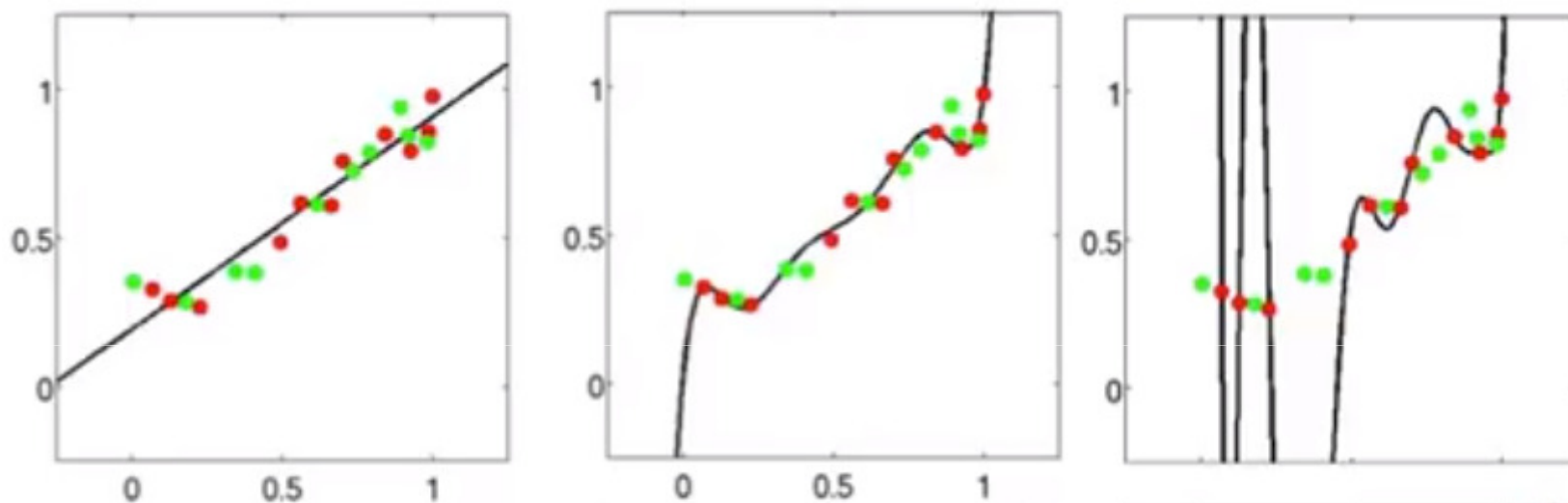
- Le λ choisi doit permettre d'obtenir un juste équilibre entre simplicité et efficacité d'apprentissage :
 - ▣ Si λ est trop élevé:
 - le modèle sera simple,
 - risque de *sous-apprentissage* des données
 - ▣ Si λ est trop faible:
 - le modèle sera plus complexe,
 - risque de *sur-apprentissage* des données



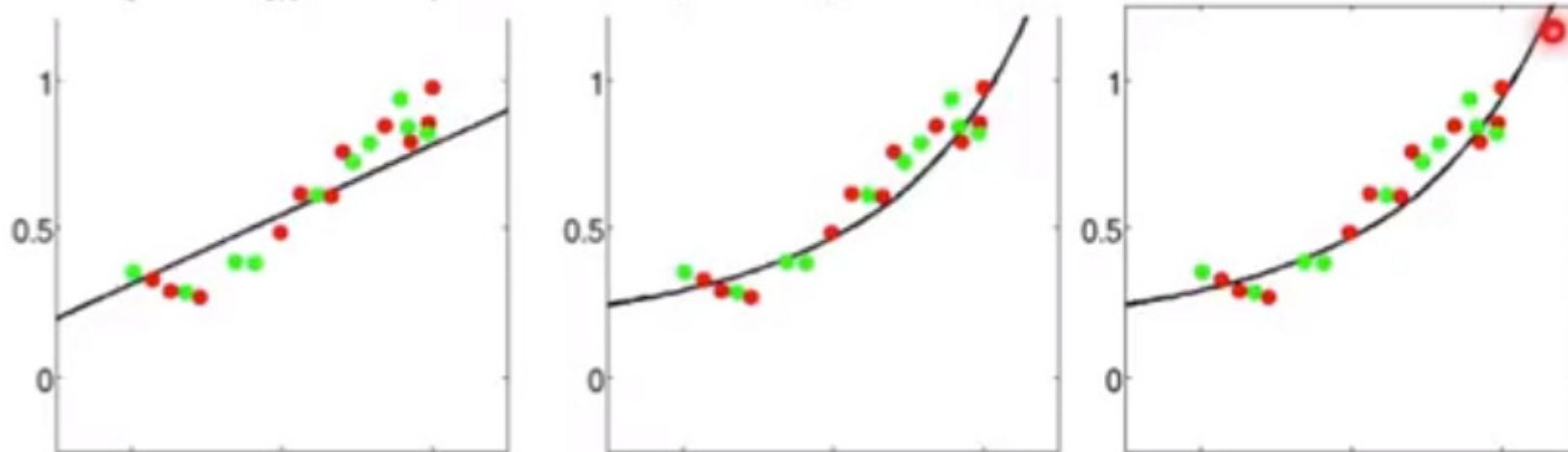
Régularisation

41

$\lambda = 0$



$\lambda = 1$



Sélection de modèle

42

- Supposons que le modèle retenu est le polynôme de degré $M = 5$:

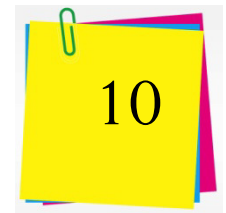
$$\theta_0 + \dots \theta_5 x^5$$

- **Question** : comment évaluer sa capacité de généralisation ?

- **Réponse** : en rapportant l'erreur $J_{\text{test}}(\theta^{(5)})$?

- **Problème** : le paramètre M a été entraîné (choisi) sur l'ensemble du test

- $J_{\text{test}}(\theta^{(5)})$ est une estimation optimale de l'erreur de généralisation



Sélection de modèle

43

- Solution 2 : Sélection du modèle avec un ensemble de validation

Size	Price		Size	Price	
2104	400		2104	400	
1600	330		1600	330	
2400	369		2400	369	
1416	232		1416	232	
3000	540		3000	540	
1985	300		1985	300	
1534	315		1534	315	
1427	199		1427	199	
1380	212				
1494	243				

D

D_{train}

D_{valid}

D_{test}

Sélection de modèle

44

■ Solution 2

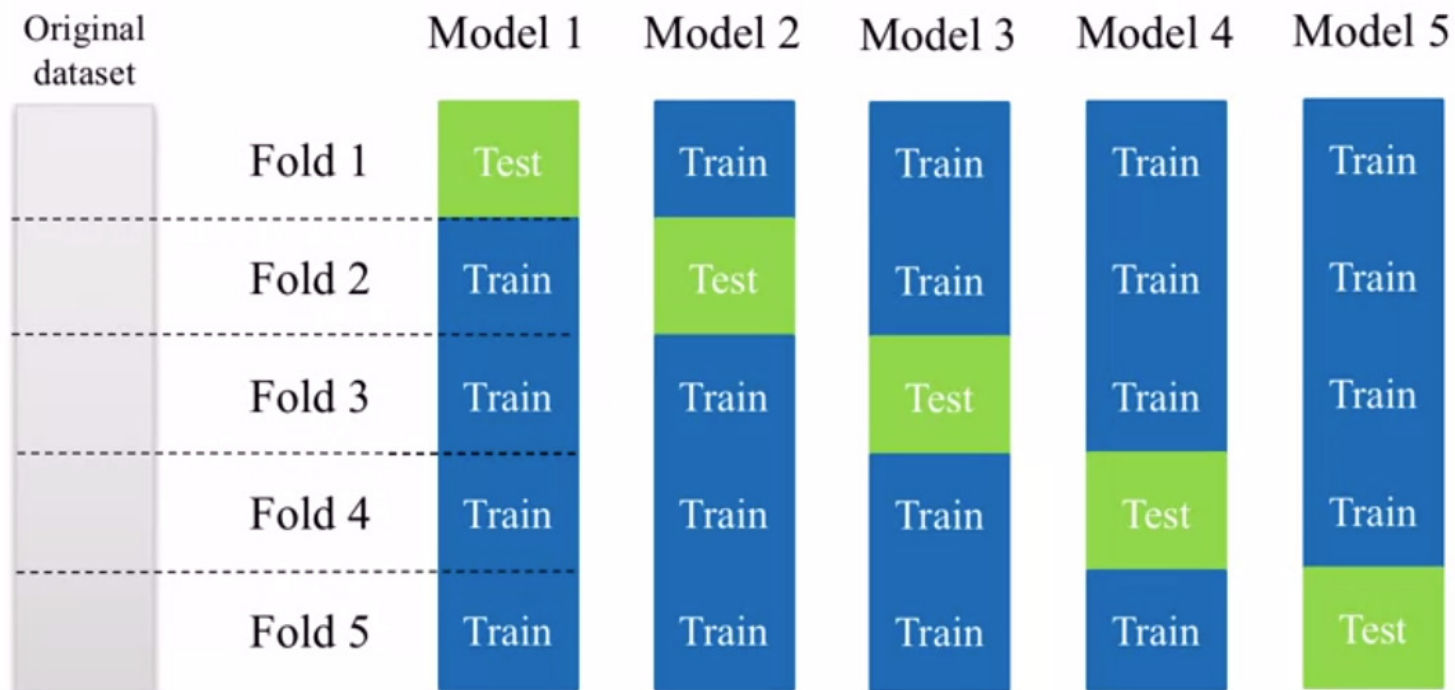
- Séparer les données en 3 ensembles: entraînement (ex. 70%), validation (ex. 15%) et test (ex. 15%)
- Faire une liste de valeurs des hyper-paramètres à essayer
- Pour chaque élément de cette liste, lancer l'algorithme d'apprentissage sur l'ensemble d'entraînement et mesurer la performance sur l'ensemble de validation
- Réutiliser la valeur des hyper-paramètres avec la meilleure performance en validation, pour calculer la performance sur l'ensemble de test
 - La performance sur l'ensemble de test est alors une estimation non biaisée (non optimiste) de la performance de généralisation de l'algorithme
 - On peut utiliser la performance pour comparer des algorithmes d'apprentissage différents

Sélection de modèle

45

- Solution 3 : Sélection du modèle avec K-fold cross-validation
 - diviser les données en K portions différentes
 - chaque portion est utilisée une fois en tant que D_{valid}

Cross-validation Example (5-fold)



Sélection de modèle

46

□ Etape 1 : Sélection de modèle avec K-fold cross-validation

▣ pour $s = 1 \dots K$

■ pour chaque valeur d'hyper-paramètres (M) à comparer

■ obtenir un modèle entraîné à partir de $D_{\text{train}} = D \setminus D_S$

■ évaluer la performance du modèle sur $D_{\text{valid}} = D_S$ en calculant l'erreur:

$$J_{cv}(\theta) = \frac{1}{m_{cv}} \sum_{i=1}^{m_{cv}} (h_{\theta}(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

■ m_{cv} : le nombre d'exemples dans D_{valid}

▣ retourner la valeur de M ayant la meilleure performance moyenne sur les ensembles D_{valid} (M qui minimise en moyenne $J_{cv}(\theta)$)

□ Etape 2 : Evaluation du modèle

▣ Estimer l'erreur de généralisation sur l'ensemble du test

■ Pour le M retenu rapporter : l'erreur $J_{\text{test}}(\theta^{(M)})$



Evaluation d'un modèle de régression linéaire

47

- ❑ Erreur absolue moyenne (MAE, pour Mean Absolute Error)

The diagram illustrates the Mean Absolute Error (MAE) formula with the following components and annotations:

- Divide by the total number of data points:** Points to the $\frac{1}{n}$ term in the formula.
- Actual output value:** Points to the y term inside the absolute value.
- Predicted output value:** Points to the $\hat{y}$ term inside the absolute value.
- Sum of:** Points to the summation symbol Σ .
- The absolute value of the residual:** Points to the absolute value bars $| \cdot |$ surrounding the difference $y - \hat{y}$.

$$MAE = \frac{1}{n} \sum \left| y - \hat{y} \right|$$

Evaluation d'un modèle de régression linéaire

48

- Erreur quadratique moyenne (MSE, pour Mean Squared Error)

$$MSE = \frac{1}{n} \sum \underbrace{\left(y - \hat{y} \right)^2}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}}$$

- **RMSE**, pour *Root Mean Squared Error* : retourne l'erreur à son échelle initiale

$$RMSE = \sqrt{\frac{1}{n} \sum \left(y - \hat{y} \right)^2}$$

Evaluation d'un modèle de régression linéaire

49

- Coefficient de détermination (R^2):
 - ▣ Le % de la variation totale décrite par la droite de régression
 - indique à quel point les valeurs prédites sont corrélées aux vraies valeurs

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

